



1.1. Knapsack Problem

"Maximize value, respect the capacity."

Main Section:

1. Greedy Approach

Contents:

[1. Problem Definition](#)

[2. A Practical Example](#)

[Summary](#)

[References](#)

The Knapsack Problem is probably one of the most famous problems in Computer Science. In fact, it is usually one of the very first problems instructors use when introducing the Greedy Algorithm—and honestly, for good reason.

The idea sounds deceptively simple. Imagine you have a bag with limited capacity, and a collection of items with different values and weights. You want to maximize what you carry, without exceeding the limit.

Simple concept. But the moment you start asking: "Which items should I take?". That's when things become surprisingly interesting. Because not every valuable item is worth taking. Some items are too heavy. Some are light but almost useless. And sometimes, choosing one item means sacrificing several better combinations later.

This is what makes the Knapsack Problem so important. It teaches us one of the biggest ideas in algorithm design: How do we make the "best" decision when resources are limited?

And depending on the version of the problem... the solution might involve Greedy Algorithms, Dynamic Programming, or even more advanced optimization techniques.

So before we jump into algorithms, let's properly define the problem first.

1. Problem Definition

Imagine you are preparing for a long journey. You have a backpack—a knapsack—but there's a problem: It can only carry a limited amount of weight.

Now suppose you also have a collection of items: Gold coins; Food supplies; Tools; Medicine; Treasures. Each item has two important properties:

- A value → how useful or important it is
- A size or weight → how much space it consumes

Naturally, you would love to carry everything. But reality says otherwise. If the total weight exceeds the knapsack's capacity, you simply cannot fit them all. So now the real challenge appears: Which items should you choose? And this is the heart of the Knapsack Problem: ***We must select a subset of items that fits within the maximum capacity, while maximizing the total value obtained.***

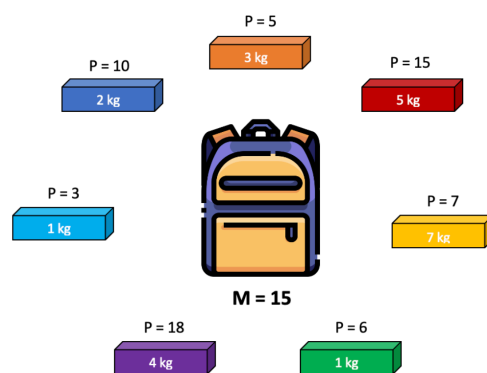
In other words: We are trying to get the greatest possible reward, using limited resources. And this is exactly why the Knapsack Problem appears everywhere in Computer Science and real life: Cargo loading; Budget allocation; Resource management; Memory optimization; Investment selection; Even game inventory systems

At its core, the problem is not really about bags or treasure. It is about decision making under constraints. And depending on how the problem is designed, solving it can become either surprisingly manageable, or incredibly difficult.

2. A Practical Example

Now let's finally see how the Greedy approach actually works in practice.

Imagine you are given a bag with a maximum capacity of 15. This means the total weight of all objects inside the bag cannot exceed 15. You are also given 7 different objects. Each object has a weight and a profit (or value). And for simplicity, each object is represented using a color.



At first glance, this might seem easy: "Just take the objects with the highest profits". But that strategy can fail very quickly. Why?

Because some objects may have high profit... but are too heavy to be worth taking.

So instead of looking only at profit, the Greedy approach asks a much smarter question:

"How much profit do we gain for every unit of weight?"

And this becomes the most important idea in the Fractional Knapsack Problem.

So in the first step, we organize everything into a table containing object, profit (P_i), weight (W_i)

And most importantly:

Profit-to-Weight Ratio:

Object	Dark Blue	Orange	Red	Yellow	Green	Purple	Light Blue
Profit	10	5	15	7	6	18	3
Weight	2	3	5	7	1	4	1
(P_i/W_i)	5	1.66	3	1	6	4.5	3

The ratio tells us how "valuable" an object is relative to its weight. For example:

- An object with profit 20 and weight 2 has a ratio of 10
- An object with profit 20 and weight 10 only has a ratio of 2

Even though both profits are the same, the first object is far more efficient.

So after computing all the ratios, we move to the second step, which is sort all objects in descending order based on their Profit-to-Weight ratio.

Object	Green	Dark Blue	Purple	Red	Light Blue	Orange	Yellow
Profit	6	10	18	15	3	5	7
Weight	1	2	4	5	1	3	7
(P_i/W_i)	6	5	4.5	3	3	1.66	1

Highest ratio first → **lowest ratio last**. This is the greedy choice. We always pick the object that gives the greatest profit for the smallest possible weight.

And now the actual selection begins. We start placing objects into the bag one by one following the sorted order:

- Include the whole green object → remaining capacity becomes: $15 - 1 = 14$
- Include the whole dark blue object → remaining capacity becomes 12.
- Include the whole purple object → remaining capacity becomes 8.
- Include the whole red object → remaining capacity becomes 3.
- Include the whole light blue object → remaining capacity becomes 2.
- The orange object has weight 3, but the bag only has remaining capacity 2. So instead of rejecting the object completely, we take only part of it.



This is the key feature of the **Fractional Knapsack Problem**: Objects are divisible. We are allowed to take fractions. So we include only: $\frac{2}{3}$ of the orange object, perfectly filling the bag.

Another Note: In the example above, ties in the Profit-to-Weight ratio did not matter because all tied objects were eventually selected. However, if two objects have the same ratio, we usually prioritize the object **with the smaller weight** because the bag capacity is limited. *For example, if the remaining capacity is 2 and two objects both have ratio 3, choosing the object with weight 2 is more practical than choosing the object with weight 5.*

- The yellow object is ignored because the bag is already full.

At the end of the process, we obtain the following sequence:

$$[2/2; 2/3; 5/5; 0/7; 1/1; 4/4; 1/1][2/2; 2/3; 5/5; 0/7; 1/1; 4/4; 1/1]$$

After simplification:

$$[1; 2/3; 1; 0; 1; 1; 1][1; 2/3; 1; 0; 1; 1; 1]$$

This sequence is extremely important. Each value tells us how much of an object is included:

- **1 → fully included; 0 → not included; Fractions → partially included**

Notice something very important:

$$0 \leq x \leq 1$$

This condition defines the Fractional Knapsack Problem. Objects can be divided into smaller parts. Examples of divisible objects: Vegetables; Fruits; Liquids; Sand. But singular objects like: Cars; Phones; Books, usually cannot be divided meaningfully. That leads to another variation called the 0/1 Knapsack Problem, which we will discuss in a future section.

Now we verify the total weight using the sequence:

$$(1 \times 2) + (2/3 \times 3) + (1 \times 5) + (0 \times 7) + (1 \times 1) + (1 \times 4) + (1 \times 1) = 15(1 \times 2) + \left(\frac{2}{3} \times 3\right) + (1 \times 5) + (0 \times$$

The bag is perfectly filled. Finally, we compute the maximum total profit:

$$(1 \times 10) + (2/3 \times 5) + (1 \times 15) + (0 \times 7) + (1 \times 6) + (1 \times 18) + (1 \times 3) = 55.3(1 \times 10) + \left(\frac{2}{3} \times 5\right) + (1 \times 15)$$

So the maximum achievable profit becomes: 55.3.

And this demonstrates the entire philosophy behind the Greedy solution: At every step, choose the object with the best profit-to-weight ratio. Take as much of it as possible. Repeat until the bag becomes full. And for the Fractional Knapsack Problem, this strategy always produces the optimal solution.

Below is the simple Python implementation of Knapsack Problem:

```
def greedy_approach(values, weights, max_weight):
    total_value = 0
    chosen_items = []
    # Sort the values using the vi/wi ratio
    items = sorted(zip(values, weights, range(len(values))), key=lambda x: x[0]/x[1], reverse=True)
    # Find the maximum total value
    for v, w, i in items:
        if max_weight >= w:
            max_weight -= w
            total_value += v
            chosen_items.append(i)
        else:
            total_value += max_weight * (v / w)
            chosen_items.append(i)
            break
    total_value = round(total_value, 1)
    return total_value, chosen_items
```

The function requires a list of values, a list of weights, and the maximum capacity of the bag. Ensure *list(values) = list(weights)*

```
# Main Function
w_list = [2, 3, 5, 7, 1, 4, 1]
v_list = [10, 5, 15, 7, 6, 18, 3]
k = 15

max_value, chosen_items = greedy_approach(v_list, w_list, k)

print(f"Maximum Value: {max_value}")
print("Chosen items:")
for d in chosen_items:
    print(f"w{d + 1}", end=" ")
```

Maximum Value: 55.3

Chosen items:

w5 w1 w6 w3 w7 w2

Summary

The Knapsack Problem is one of the most classic and important optimization problems in Computer Science.

At its core, the problem teaches a very important lesson: Resources are limited. So we must make intelligent decisions about what to include, and what to sacrifice.

In the Fractional Knapsack Problem, the Greedy Algorithm works perfectly because objects are divisible. Instead of blindly choosing the largest profits, we choose objects based on their Profit-to-Weight ratio. This allows us to maximize value while minimizing wasted capacity.

We also saw the complete process:

Calculate ratios → Sort objects → Select greedily → Take fractions if necessary → Compute the final maximum profit

So while the problem itself may look simple, the underlying idea is incredibly powerful. Because many real-world systems—from budgeting and resource allocation to cargo loading and memory optimization—follow this exact same philosophy:

Choose the option that gives the greatest benefit for the least possible cost.

References

https://en.wikipedia.org/wiki/Knapsack_problem

<https://www.geeksforgeeks.org/dsa/fractional-knapsack-problem/>

<https://www.youtube.com/watch?v=oTTzNMHM05I>